

Contract Fragility: An Empirical Evaluation Protocol for Schema Complexity and Structural Degradation in LLM Tool Calling

Rishindra Mateti

Abstract—Tool-augmented Large Language Models (LLMs) and autonomous agent architectures depend fundamentally on structured JSON Schema contracts to interface with external execution environments. While recent research has extensively addressed context-window exhaustion through tool retrieval and schema compression, the causal influence of tool contract design—specifically object nesting depth, optionality density, enum constraint specificity, and identifier ambiguity—remains an unstandardized empirical question. In current software engineering practice, developers author schemas through heuristic intuition, frequently introducing structural friction that induces parameter omission or schema validation rejection. This paper establishes a formal empirical benchmark protocol to evaluate contract fragility across 20 synthetic API-inspired tool specifications modeled after production REST and Model Context Protocol (MCP) standards. We propose a decoupled three-task evaluation architecture: (1) Argument Construction under isolated parameter complexity without distractors, (2) Tool Selection under domain-matched competing distractor suites with randomized presentation order, and (3) Execution Semantics evaluating recursive field-level grounding. We outline concrete structural failure hypotheses, formalize statistical evaluation metrics based on Wilson score confidence intervals and McNemar paired exact tests, and introduce an open-source static schema linter (*SchemaFragilityLinter*) to audit tool contracts prior to agentic deployment.

Index Terms—Function Calling, Tool-Augmented LLMs, JSON Schema, Agentic Systems, Model Context Protocol, Empirical Software Engineering, Benchmark Protocol.



1 INTRODUCTION

FUNCTION calling and autonomous agent architectures have transitioned into foundational building blocks of modern software engineering. Frameworks such as Anthropic’s Model Context Protocol (MCP) [?], Google Agent Development Kit, and OpenAI Function Calling enable Large Language Models (LLMs) to query databases, invoke APIs, and trigger mutations across enterprise environments.

Central to every function-calling interface is the *tool contract*: a structured specification encoded in JSON Schema (Draft-07), defining the function identifier, semantic descriptions, parameter types, enums, and required properties. When presented with a task, the model must map natural language intent into valid, strictly conforming JSON arguments matching the target schema.

Despite the critical role of schemas in agent reliability, tool-contract authoring in industry remains largely unguided by empirical standards. Developers routinely transplant raw backend object models directly into tool definitions. While recent literature has tackled context-window bloat through tool retrieval [?], [?], [?], the software engineering community lacks a systematic measurement of how *internal contract structure* causally impacts model execution.

Benchmarks such as the Berkeley Function Calling Leaderboard (BFCL) [?] evaluate model performance across static, heterogeneous schemas to rank models. However, they do not causally perturb schema properties to isolate which specific contract design choices cause failure. While recent

empirical findings demonstrate that wording changes in tool docstrings can alter model selection probabilities by over an order of magnitude [?], the causal effects of object nesting, enum constraints, and optional field density remain unquantified.

1.1 Contributions

This paper establishes a controlled empirical evaluation protocol for contract fragility:

- 1) **A Three-Task Decoupled Benchmark Architecture:** We decouple function-calling evaluation into three distinct, orthogonal tasks: (Task 1) Argument Construction with isolated parameter complexity, (Task 2) Tool Selection with randomized domain-matched distractor suites, and (Task 3) Execution Semantics measuring recursive field-level value grounding.
- 2) **A Controlled Structural Mutation Taxonomy:** We formulate 80 controlled schema conditions derived from 20 synthetic API-inspired specifications spanning four core structural mutations: canonical flat baseline, nested hierarchy, optionality bloat, and ambiguous identifiers.
- 3) **Transparent Dataset Provenance:** We provide a fully documented manifest of synthetic schemas modeled after enterprise API patterns (Stripe, GitHub, Slack, Kubernetes, AWS S3, etc.) ensuring reproducibility without operational security risks.
- 4) **Static Contract Linter:** We introduce *SchemaFragilityLinter*, an open-source static

• R. Mateti is a Software Engineer based in Dallas, TX, USA.
E-mail: contact@rishindramateti.com — ORCID: 0009-0009-5880-8727

analyzer that audits JSON Schema definitions for structural fragility anti-patterns prior to deployment.

2 RELATED WORK

2.1 Function Calling Benchmarks and Model Capabilities

The evaluation of LLMs as tool users has evolved from simple single-turn API matching to complex agent environments. The Berkeley Function Calling Leaderboard (BFCL) [?] represents the standard comprehensive benchmark for model comparison across Python, REST, and SQL interfaces. Tool-Bench [?] introduced 16,000 real-world APIs to benchmark tool retrieval and multi-step execution. These benchmarks treat schemas as fixed artifacts to evaluate models, whereas our study holds models fixed to evaluate *schema design properties*.

2.2 Tool Retrieval and Context Compression

As tool catalogs expand, context-window overflow has become a primary bottleneck. Recent systems propose diverse retrieval mechanisms: AnyTool [?] introduced a hierarchical API retriever, while ToolkenGPT [?] modeled tools as specialized embedding tokens. Sakizli [?], [?] demonstrated that standard JSON Schema definitions overflow small context budgets, proposing deterministic token compilation (TSCG) to compress schemas by 44–57%. In parallel, Babu and Iyer [?] introduced Causal Minimal Tool Filtering (CMTF) to expose minimal next-step tools. Our work investigates the complementary question: *for the tools present in the context, how does structural contract design govern reliability?*

2.3 Prompt Sensitivity and Wording Biases

Recent natural language processing research highlights the sensitivity of LLMs to minor prompt phrasing. Zhang et al. [?] demonstrated that small wording modifications to tool docstrings drastically alter selection behavior. This protocol extends that inquiry from natural language descriptions to formal structural schema properties.

3 THE THREE-TASK EVALUATION TAXONOMY

To avoid conflating tool retrieval with parameter extraction and semantic correctness, our protocol isolates model behavior across three orthogonal tasks (Fig. 1):

3.1 Task 1: Argument Construction

In Task 1, exactly one correct tool is provided in the prompt, eliminating selection ambiguity. The model is presented with a frozen user query and must construct arguments conforming to the provided schema. Outputs are deterministically validated using JSON Schema Draft-07 validation. Failure modes are categorized into:

- **MISSING_REQUIRED_FIELD:** Omission of mandatory contract parameters.
- **TYPE_MISMATCH:** Parameter generated with invalid type (e.g., string instead of integer).
- **ENUM_VIOLATION:** Value generated outside allowable enum constants.
- **UNEXPECTED_FIELD:** Generation of unrequested parameters not defined in schema.

3.2 Task 2: Tool Selection

In Task 2, the target tool is embedded alongside domain-matched, plausible distractor tools. To prevent positional bias, the presentation order of tools in the prompt menu is randomized with a deterministic seed. We evaluate whether tool-level naming and description ambiguity degrade Top-1 tool selection accuracy.

3.3 Task 3: Execution Semantics

Task 3 evaluates outputs that successfully pass syntactic schema validation. A payload may be structurally valid according to JSON Schema but semantically ungrounded. We implement recursive field-by-field evaluation comparing flattened paths, measuring Exact Match (EM), Field Precision, and Field Recall against human-verified ground-truth arguments.

4 DATASET SPECIFICATION & MUTATION TAXONOMY

4.1 Synthetic Specification Corpus

The dataset comprises 20 synthetic tool specifications modeled after production REST and MCP API conventions across four industry sectors:

- 1) **Fintech & Billing:** Stripe refunds, customer invoicing, payment captures.
- 2) **Developer Tools & Cloud:** GitHub issue tracking, Kubernetes deployment scaling, AWS S3 provisioning, Docker runtime execution.
- 3) **Enterprise CRM & Communications:** CRM lead status routing, Zendesk ticket triage, Twilio SMS dispatch, Slack alerting.
- 4) **Infrastructure & Observability:** Datadog monitoring alerts, PagerDuty incidents, Cloudflare CDN cache invalidation, Elasticsearch indexing.

4.2 Controlled Structural Mutations

For each base schema, we programmatically construct four controlled variants holding user intent frozen:

- **Canonical (Baseline):** Clean, flat parameter hierarchy with explicit domain nouns and complete docstrings.
- **Nested Hierarchy:** Parameters are encapsulated into a nested sub-object (`request_payload`), introducing hierarchical depth.
- **Optionality Bloat:** Six optional enterprise metadata properties (`tags`, `correlation_id`, `idempotency_key`, `priority_level`, etc.) are appended.
- **Ambiguous Identifiers:** Explicit names are replaced with truncated abbreviations and generic tokens (`process_action`, `target_id`, `qty`, `val`) across both tool-level and parameter-level descriptors.

Task 1: Argument Construction (Zero Distractors)

Input: User Query + Exactly One Target Schema (Canonical, Nested, Bloated, or Ambiguous).

Evaluation: Strict JSON Schema Draft-07 validation via `Draft7Validator`. Metric: Syntactic Validation Pass Rate (SIR = 1 – Pass Rate).

Task 2: Tool Selection (Competing Distractor Suites)

Input: User Query + Target Schema + Domain-Matched Distractor Tools (Randomized Menu Order).

Evaluation: Top-1 Tool Selection Accuracy (%) comparing Canonical vs. Ambiguous names, controlling for positional bias.

Task 3: Execution Semantics (Recursive Field-Level Value Grounding)

Input: Syntactically Valid Outputs from Task 1.

Evaluation: Recursive field-by-field Exact Match (EM), Precision, and Recall against ground-truth values.

Fig. 1. The Three-Task Decoupled Evaluation Framework for Contract Fragility.

4.3 Mutation Isolation Verification

To prevent confounding interactions between structural dimensions, we implement a formal property-based unit test suite (`tests/test_mutations.py`). For every specification across the corpus, our test harness asserts that:

- **Nesting Isolation:** The `nested_hierarchy` transformation introduces strictly one root property (`request_payload`) while maintaining exact invariance of sub-property types, descriptions, and enum sets.
- **Optionality Isolation:** The `optionality_bloat` transformation preserves all required parameters without modification, appending exclusively optional metadata properties.
- **Ambiguity Isolation:** The `ambiguous_identifiers` transformation alters parameter and tool names while strictly preserving underlying JSON types, required status, and enum domains.

5 EXPERIMENTAL PROTOCOL AND TARGET HYPOTHESES

5.1 Testable Research Hypotheses

The benchmark protocol is designed to test three foundational hypotheses:

- **H1 (Nesting Fragility):** Encapsulating flat parameters into nested sub-objects significantly increases schema validation failure rates, primarily driven by parameter flattening at the root level.
- **H2 (Identifier Disambiguation):** Truncating tool names and parameter descriptors causes statistically significant degradation in Top-1 tool selection accuracy in multi-tool contexts.
- **H3 (Optionality Clutter):** Appending high-density optional parameters inflates prompt token overhead without improving task success, inducing hallucination of default arguments.

5.2 Threats to Validity: Provider-Mediated Interfaces

A fundamental methodological distinction in LLM tool-calling research is the difference between raw transformer attention and provider-mediated execution. Commercial

endpoints (such as Google GenAI, OpenAI Function Calling, and Anthropic Tools) do not expose raw token logits; they apply server-side schema compilation, JSON grammar masks (e.g., Outlines-style finite state machines), and proprietary system prompts. Consequently:

- 1) **Scoping of Claims:** Findings on commercial APIs must be reported as characteristics of the *provider-mediated interface*, not general unconstrained LLM attention over schemas.
- 2) **Protocol Disentanglement:** Our protocol specifies two evaluation tracks: (Track A) vendor-native tool calling with provider-enforced schemas, and (Track B) unconstrained raw prompting with temperature sampling and standard JSON extraction, isolating raw generative competence from vendor-side schema repair.

5.3 Statistical Methodology & Smoke Test Disclosure

To ensure mathematical rigor, all evaluations enforce:

- **Wilson Score Confidence Intervals:** 95% confidence intervals computed for all binary proportions.
- **McNemar’s Paired Exact Tests:** Evaluated on discordant pairs between baseline and mutated conditions to determine statistical significance ($p < 0.05$).
- **Infrastructure Separation:** HTTP availability failures (e.g., 429 throttling) are strictly isolated from model accuracy scoring.

Smoke Test Disclosure: The initial baseline telemetry committed to the repository represents a single-suite pipeline smoke test ($N = 1$ suite, 6 API calls on `gemini-3.1-flash-lite-preview`) to verify end-to-end execution, schema parsing, and statistical calculation pipelines. The wide resulting 95% Wilson intervals ([20.7%, 100.0%]) and lack of discordant pairs ($p = 1.0$) illustrate that this run serves exclusively as harness verification, not empirical proof of model capabilities. Full empirical execution requires our planned multi-provider sweep across 20+ schemas and 3–5 independent repetitions across Gemini, GPT-4o, Claude 3.5 Sonnet, and open-weights Llama-3.1-70B.

6 STATIC SCHEMA LINTING (SCHEMAFRAGILITYLINTER)

As an engineering artifact, we release `SchemaFragilityLinter`, a static analysis utility

that audits JSON Schema contracts for candidate structural risks identified by this protocol prior to deployment:

- `DEEP_HIERARCHICAL_NESTING`: Flags nested object hierarchies exceeding 1-level depth.
- `AMBIGUOUS_IDENTIFIER`: Flags generic or truncated parameter names (`id`, `data`, `val`).
- `EXCESSIVE_OPTIONALITY_BLOAT`: Flags schemas where optional properties exceed required properties by more than 2:1.
- `MISSING_PARAMETER_DOCSTRING`: Flags parameters lacking semantic documentation.

7 CONCLUSION AND NEXT STEPS

This paper establishes the formal specification and evaluation protocol for contract fragility in LLM function calling. By decoupling evaluation into argument construction, tool selection, and execution semantics, this framework provides a standardized testbed to assess how schema design choices govern reliability. The full benchmark testbed, dataset, mutation isolation tests, and static linter are open-sourced to facilitate reproducible agent engineering.